



LPC 2026 • LIVEPATCHING MICROCONFERENCE

Ftrace Consumer Classification & Livepatch Coexistence

Empirical klp Contender & Incumbent Coexistence Across All Ftrace Users

18 Ftrace Consumers • 3 Behavioral Classes • 28 klp Scenarios + 11 Order Tests (Linux 7.2)



LINUX
PLUMBERS
CONFERENCE

PRAGUE, CZECHIA / OCT. 5-7, 2026

1. Ftrace Behavioral Classes: Declared Flags vs. Actual Capability

Ftrace conflict checks only inspect IPMODIFY and DIRECT flags – not actual runtime behavior.

Behavioral Decision Procedure

Q1: Does it set `FTRACE_OPS_FL_IPMODIFY`
(or otherwise write `regs->ip`)?

→ YES: Class 1 – Mutator

Q2: Can it skip `orig_func` or replace its
return value (`BPF_TRAMP_MODIFY_RETURN`)?

→ YES: Class 1 – Mutator

Q3: Does it install a return hook
(`rethook` / `fgraph` / `direct fexit stub`)?

→ YES: Class 2 – Stack Interceptor

Otherwise:

→ Class 3 – Passive Observer

Class 1 – Mutator (A–E)

- **Diverts control flow:** alters `regs->ip`, skips `orig_func`, or overrides return value.
- **Users:** `kdp`, `fail_function`, `bpf_override_return`, `BPF fmod_ret`, `kprobe+post`.

Class 2 – Stack Interceptor (F–I)

- **Preserves `orig_func`,** but replaces stack return address with exit stub.
- **Users:** `BPF fexit`, `fgraph`, `kretprobe` / `rethook`, `fprobe+exit`.

Class 3 – Passive Observer (J–N)

- **Read-only at entry:** never modifies `regs->ip`, return value, or stack.
- **Users:** `perf`, `BPF fentry`, `fprobe (entry)`, `kprobe (entry)`, `function tracers`.



LINUX
PLUMBERS CONFERENCE

PRAGUE, CZECHIA / OCT. 5-7, 2026



2. Kernel Ftrace Consumer Inventory: Class 1 & Class 2 (A–I)

Class 1 Mutators (A–E) divert control flow; Class 2 Stack Interceptors (F–I) hook the return address on the stack.

Class	Consumer / Subsystem (ID)	Ftrace Mechanism / Flags	Runtime Behavior & Impact on Function
1 – Mutator	A: Kernel Livepatching (klp)	ftrace_ops (IPMODIFY)	Redirects regs->ip to new_func; skips orig_func.
1 – Mutator	B: Error injection (fail_fn)	kprobe_ipmodify_ops	Short-circuits body via just_return_func (-EIO).
1 – Mutator	C: BPF bpf_override_return	kprobe_ftrace_ops (no I)	Short-circuits (-EINVAL) without setting IPMODIFY.
1 – Mutator	D: BPF fmod_ret / BPF LSM	ftrace_direct (MODIFY_RET)	JIT skips orig_func on non-zero return (-EPERM).
1 – Mutator	E: Kprobes (with post_handler)	kprobe_ipmodify_ops	Emulated single-step, ip restored; Class 3 w/o it.
2 – Interceptor	F: BPF trampoline fexit	ftrace_direct (CALL_ORIG)	Wraps return via exit stub; shares IPMODIFY.
2 – Interceptor	G: Function graph (fgraph)	graph_ops + return_handler	Hijacks stack return address to measure duration.
2 – Interceptor	H: kretprobes / rethook	kprobe_ftrace_ops + rethook	Replaces stack[0] with arch_rethook_trampoline.
2 – Interceptor	I: Fprobe (with exit_handler)	fprobe_graph_ops (fgraph)	Wraps entry/exit via fgraph (exit_handler != NULL).

2. Kernel Ftrace Consumer Inventory: Class 3 (J–N)

*Class 3 Passive Observers read entry state only (*bottom rows: N's ftrace_ops; profiler uses fgraph → Class 2).*

Class	Consumer / Subsystem (ID)	Ftrace Mechanism / Flags	Runtime Behavior & Impact on Function
3 – Observer	J: Plain ftrace_ops	ftrace_ops (no flags)	Read-only entry callback; not PMU sampling.
3 – Observer	K: BPF trampoline fentry	ftrace_direct (fentry)	Read-only JITed argument inspection at entry.
3 – Observer	L: Fprobe (no exit_handler)	fprobe_ops (SAVE_ARGS)	Batch entry probing for BPF kprobe_multi.
3 – Observer	M: Kprobes (no post_handler)	kprobe_ftrace_ops (REGS)	Inspects pt_regs arguments at entry (no IPMODIFY).
3 – Observer	N: Core function tracer*	trace_ops (function)	Logs function entry events into trace ring buffer.
3 – Observer	*Latency tracers (irqsoff)	trace_ops or fgraph	Measures irq-disabled and wakeup latency.
3 – Observer	*Stack tracer (stack_tracer)	stack_ops (trace_stack.c)	Checks stack depth at entry for max stack usage.
2 – Interceptor	*Function profiler (ftrace_profile)	fgraph (fprofiler_ops)	Call counts and durations via fgraph return hook.
3 – Observer	*Persistent store (pstore)	pstore_ftrace_ops	Logs function entries to NVRAM for crash analysis.

3. Empirical klp Coexistence & Contention (Class 1 / 2 / 3)

Peer (A-I)	Flags	klp Contender	klp Incumbent
A: klp	1:I / 1:IM	Self	Self
B: fail_function	1:RI / 1:RIM	-EBUSY	-EBUSY
C: bpf_override	1:R / 2:RIM	Suppressed	Suppressed
D: BPF fmod_ret	1:RD / 2:RIDM	Suppressed	Override
E: kprobe+post	1:RI / 1:RIM	-EBUSY	-EBUSY
F: BPF fexit	1:RD / 2:RIDM	OK	OK
G: fgraph	1:none / 2:IM	Stale Symbols	Stale Symbols
H: kretprobe	1:R / 2:RIM	OK	OK
I: fprobe+exit	1:none / 2:IM	OK†	OK

Flags (/sys/kernel/tracing/enabled_functions <count>:<flags>):
 • R = FTRACE_FL_REGS (pt_regs) • I = FTRACE_FL_IPMODIFY (regs->ip)
 • D = FTRACE_FL_DIRECT (tramp) • M = FTRACE_FL_MODIFIED (sticky, omitted solo)

Peer (J-N)	Flags	klp Contender	klp Incumbent
J: ftrace_ops	1:none / 2:IM	Stale Symbolst	Stale Symbols
K: BPF fentry	1:RD / 2:RIDM	Stale Symbols	Stale Symbols
L: fprobe (entry)	1:none / 2:IM	Stale Symbolst	Stale Symbols
M: kprobe (entry)	1:R / 2:RIM	Stale Symbols	Stale Symbols
N: function tracer*	1:none / 2:IM	Stale Symbols	Stale Symbols

Cell States

- **Self (A)**: 2nd livepatch's new_func2 wins (same klp ftrace_ops)
- **-EBUSY (B, E)**: Contender rejected; Incumbent keeps behavior
- **Override (D, klp Incumbent)**: D skips new_func (-EPERM) in either order
- **Suppressed (C)**: older ftrace_ops runs last and its regs->ip wins
- **Stale Symbols (G, J-N)**: klp runs new_func; peer files it under orig_func
- **OK (F, H, I)**: return hook sees new_func's return value (ret=7)
- **† (I, J, L)**: peer ops older than klp, sees fregs ip = new_func

4. Three Livepatch Coexistence Gaps

Gap 1 — Undeclared regs->ip Mutation: C (bpf_override_return) vs. A (klp) → LIFO Priority

- **Mechanism:** C writes `regs->ip = just_return_func` via `kprobe_ftrace_ops`, which never declares `FTRACE_OPS_FL_IPMODIFY`.
- **Impact:** Ftrace admits both (2:RIM). The LIFO list runs the older ops last, so it wins — yet older ops ≠ first consumer: an unrelated `kprobe` makes C win; reloading `klp` flips it.

Gap 2 — DIRECT Trampoline Bypasses IPMODIFY: D (BPF fmod_ret) vs. A (klp) → DIRECT Priority

- **Mechanism:** D registers via `register_ftrace_direct()` and shares the exact same `bpf_trampoline_ops_func()` `SHARE_IPMODIFY` callback as benign BPF `fexit` (F).
- **Impact:** Ftrace grants coexistence (2:RIDM). Because `call_direct_funcs()` executes AFTER `ftrace_ops_list`, D's trampoline branches past `call *rbx` (-EPERM) and defeats `klp` in both attachment orders.

Gap 3 — Symbol Misattribution to orig_func: Class 2 (F-I) & Class 3 (J-N) vs. A (klp) → IPMODIFY Priority

- **Mechanism:** No consumer F-N is told about the redirect; all file the entry under `orig_func` (`bpf_get_func_ip / kp.addr / entry_ip`). Return hooks F, H and I still see `new_func`'s return value.
- **Impact:** Entry events land on `orig_func` although its body never runs, while PMU (Performance Monitoring Unit) cycles land in `new_func`. `SHARE_IPMODIFY` does not fix this: H and I match F without it.

5. Path Forward: User Defined Priority & Spatial Decoupling

Gap	Today	User Defined Priority & Spatial Decoupling
Gap 1 — Undeclared regs->ip Mutation and -EBUSY	• C writes regs->ip on kprobe_ftrace_ops without IPMODIFY; older ftrace_ops wins (flips on reload). • B & E are rejected with -EBUSY on orig_func.	• Replace LIFO with User Specified Priority / Policy. • Spatial Decoupling (new_func): Attach B / C / E to new_func — no IPMODIFY conflict with klp; error injection & post_handler work cleanly.
Gap 2 — DIRECT Trampoline Bypasses IPMODIFY	• Shares bpf_trampoline_ftrace_ops_func() SHARE_IPMODIFY with benign BPF fexit (F). • DIRECT trampoline runs after ftrace_ops_list and skips new_func (-EPERM).	• Gate SHARE_IPMODIFY & Priority (orig_func): Do not auto-accept SHARE_IPMODIFY; enforce user-defined priority. • Spatial Decoupling (new_func): May coexist by attaching SHARE_IPMODIFY to new_func (1:RDM).
Gap 3 — Symbol Misattribution to orig_func	• F–N get no redirect notification and stay bound to orig_func. • F, H, I report new_func's return value but file the entry under orig_func; PMU cycles land in new_func.	• Spatial Decoupling (new_func): Export and trace new_func directly across all Class 2 & Class 3 consumers (F–N). • Coherent Attributes: per-call fregs ip check if IPMODIFY runs first; else notifiers